CASE-STUDY PRACTICE | CAPSTONE 1 | STUDY EDITION

Visual Search: One Embedding for Three Products

Scenario questions on replacing three specialized image embeddings with one shared multi-task embedding, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on a GPU, a database or a search index: figures are the paper's or arithmetic from stated inputs, and corpus size, hardware speeds and prices are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: unification, evidence and launch decisions 6

2 The model: shared backbone, classification heads and binary embeddings 10

3 The index and the query path: storage, shards and search 14

4 Evaluation, operations, risk and scale 19

Sources and confidence 23

Every question, by part

Part 1 Framing the problem: unification, evidence and launch decisions 6

01 Three embeddings, or one? Framing 6

02 Was it multi-task training, or just a better backbone? Evidence 7

03 Offline says +0.6, human raters say +110% Evaluation 8

04 Launch Shop-the-Look without an A/B test? Launch decision 9

Part 2 The model: shared backbone, classification heads and binary embeddings 10

05 Can a classifier train an embedding? Mechanism 10

06 A noisy head in a shared trunk Debugging 11

07 Float, low-dimensional float, or binary? Design choice 12

08 Equal loss weights and uniform dataset mixing Training design 13

Part 3 The index and the query path: storage, shards and search 14

09 Size the index Sizing 14

10 Review the per-shard scan Code review 15

11 Add a 17th shard Resharding 16

12 Fan-out makes the tail worse Scatter-gather 17

13 How many replicas at peak? Capacity 18

Part 4 Evaluation, operations, risk and scale 19

14 A retrain ships and binary quality drops Operations 19

15 Can you trust the evaluation set? Evaluation 20

16 Camera photos as queries: privacy and abuse Risk 21

17 Ten times the images and the traffic Scale 22

The case in one page

A summary of the case study to read before the questions.

Problem	Three visual search products each had their own embedding model, training pipeline and backfill. Every model improvement meant re-embedding billions of images, three times over.
Core idea	One shared SE-ResNeXt101 backbone trained on all three products' data through four classification heads gives one unified embedding, which is then binarized to one bit per dimension.
Metric	Retrieval precision on each product's own evaluation set: P@1 for Shop-the-Look, average P@20 for Flashlight and Lens. The question is whether one embedding matches or beats three specialists.
Headline results	Unified vs old specialized: Shop-the-Look P@1 52.8 vs 33.0, Flashlight 60.2 vs 53.4, Lens 18.4 vs 17.8 (source-confirmed).
What multi-task alone adds	Same architecture trained on one dataset vs all three: +3.6 Shop-the-Look, +6.8 Flashlight, +0.2 Lens (source-confirmed). The rest of the gain over the old models came from architecture and data.
Binarization	A naive binary embedding lost 5.6, 4.5 and 0.9 points against float. Four architecture changes recovered the loss (source-confirmed).
Scale and storage	Over 600 million searches a month, billions of Pins (source-confirmed). Illustrative: 5 billion images, 2,048 dimensions, 1.28 TB binary against 40.96 TB float, 16 shards by capacity.
Easy-to-miss traps	Credit given to multi-task training that belongs to the new backbone. A scan that needs a 640 GB temporary array. A shard count chosen by capacity when throughput decides. Modulo sharding that moves 94% of rows. Fan-out that makes the tail worse.

Numbers to know

Each figure carries the tag that says how it was obtained. The corpus size, hardware speeds and prices are illustrative.

Quantity	Value	Evidence
Old specialized embeddings	Flashlight 4096-dim (VGG16 FC6), Lens 2048-dim (ResNeXt50), Shop-the-Look 2048-dim (ResNet101)	source-confirmed (the paper)
Training data	Flashlight about 800K images, 15K classes; Lens 540K, 2K; Shop-the-Look 340K, 189 product classes, 50K instance labels	source-confirmed (the paper)
Production scale	Over 600 million visual searches a month; over 250 million monthly active users; billions of Pins	source-confirmed (the paper)
Average and peak queries	600,000,000 ÷ 2,592,000 s = 231.5/s; peak about 926/s at an assumed 4 times	illustrative; recomputed
Index size	2,048 × 4 B = 8,192 B float; 256 B binary; 5 billion images: 40.96 TB vs 1.28 TB (32 times)	illustrative; recomputed
Shards	1.28 TB ÷ 100 GB = 12.8, so 13; with 20% headroom 15.36, so 16	illustrative; recomputed
Backfill	5 billion ÷ 1,000 per GPU-s = 1,389 GPU-hours = $3,472 at $2.50; three models $10,417	illustrative; recomputed
Table 4: unified vs old	Shop-the-Look P@1 52.8 vs 33.0; Flashlight 60.2 vs 53.4; Lens 18.4 vs 17.8	source-confirmed (the paper)
Table 3: multi-task ablation	Alone vs all: Shop-the-Look 49.2 to 52.8; Flashlight 53.4 to 60.2; Lens 18.2 to 18.4	source-confirmed (the paper)
Table 1: float vs binary	Float baseline 47.5, 60.1, 18.6; binary baseline 41.9, 55.6, 17.7; final binary 52.8, 60.2, 18.4	source-confirmed (the paper)
Evaluation sets	Shop-the-Look 600 objects, 1,421 matches, corpus 51,421; Flashlight 807 queries, 42,881; Lens about 1K, about 49K	source-confirmed (the paper)
Noise of a P@1	52.8% on 600 queries: margin about plus or minus 4.0 points	recomputed
Human judgment (P@5)	Flashlight +22.2%, Lens +110.1%, Shop-the-Look 37.3% to 64.2% (+72.1%)	source-confirmed (the paper)
Online test, Lens	Close-ups +32.7%, repins +46.7%, clickthroughs +35.0%	source-confirmed (the paper)
Exact scan of one shard	80 GB at 100 GB/s = 0.8 s; about 740 replicas per shard, 11,850 machines at 926/s	illustrative; recomputed
Sharding change	16 to 17 shards: modulo moves 94% of rows, consistent hashing 6%	recomputed
Fan-out tail	Shard p99 100 ms: 15% of queries slower with 16 shards, 79% with 154	illustrative; recomputed
Ten times the load	50 billion images: 12.8 TB binary, 154 shards, 9,259 searches/s, backfill 13,889 GPU-hours = $34,722	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
One embedding or three	One shared multi-task embedding	A head's metric falls as others are added: split it or use adapter heads	source-confirmed; reasoned
Embedding format	Binarized, with an architecture adapted to it	Float or low-dimensional float if binary loses too much on your data	source-confirmed; reasoned
Loss weights and mixing	Equal weights, uniform mixing	Learned weights if one task dominates or lags	source-confirmed
Shard key	Consistent hashing	Plain modulo only if the shard count will never change	reasoned
Per-shard search	Approximate index over binary codes	Exact scan for small shards or for re-ranking a short list	reasoned
Query fan-out	All shards, with hedging for the tail	A subset of shards only with a measured recall loss	reasoned
Launch gate	A/B test where traffic allows	Human judgment with a staged rollout where it does not	source-confirmed; reasoned

PART 1

Framing the problem: unification, evidence and launch decisions

Deciding whether one embedding should replace three, reading the evidence for it honestly, and choosing how to launch.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Embedding	A vector of numbers that a model produces for an image, placed so that similar images are close together. Search finds the stored vectors nearest to the query's vector.
Backfill	Re-computing the embedding of every image in the corpus and re-writing the index, which every new embedding model requires. With billions of images it is a large, recurring cost.
Precision at k (P@1, P@5, avg P@20)	Of the top k results for a query, the share that are relevant. P@1 looks at the single top result. Avg P@20 averages the top-20 share over a set of queries.
Ablation	An experiment that removes or changes one part of a system to measure what that part contributes. Training on one dataset alone, instead of all three, is an ablation of multi-task training.
Negative transfer	Training a model on an extra task or dataset that makes it worse at another task, instead of merely not helping.
Offline and online evaluation	Offline evaluation scores a model on a fixed labeled set before launch. Online evaluation measures what real users do after exposure, usually in an A/B test.
A/B test	Showing different versions to randomly split groups of users at the same time and comparing a behavior metric, such as clicks. It gives a causal read on user behavior.
Worker consistency	The paper's measure of how often the same judging question gets the same answer across repeated jobs. It checks the reliability of the judging process, not agreement between different raters.
Where to read	Zhai et al.: Learning a Unified Embedding for Visual Search at Pinterest

01 Three embeddings, or one?

FRAMING

SCENARIO

A company runs three visual search products. Each has its own embedding model, training pipeline and backfill, over about 5 billion images. Leadership proposes replacing them with one shared embedding. The Lens team objects that their metric will suffer.

HOW TO THINK ABOUT IT

Price both sides before taking one. List what unification saves, what it adds, and what evidence would show that the objection is wrong.

Q What does each design cost?

A Separate embeddings cost one backfill per model. Assume one GPU embeds 1,000 images a second at $2.50 an hour (illustrative).

- One backfill: 5,000,000,000 ÷ 1,000 = 5,000,000 GPU-seconds = 1,389 GPU-hours, about **$3,472**.
- Three specialized models refreshed together cost about $10,417. One unified model costs $3,472.
- The ratio does not depend on the throughput guess: three pipelines mean three backfills.

Q What does the paper say was the pain?

A Each improvement in embedding required a full backfill that could be prohibitively expensive (source-confirmed).

- It adds that downstream systems depended on specific versions of each embedding, so the company kept producing several generations of the same specialized embeddings.

Q What would you need to see before agreeing with unification?

A Per-product metrics at least equal to today's, on each product's own evaluation set.

- Run the ablation in Question 2: train on each dataset alone and on all together, with the same architecture.
- Add human judgment on live queries, and a staged rollout (Question 4).

Q What does unification not remove?

A A full backfill is still needed for each improvement, but once, not three times.

- Negative transfer remains a risk: one product's labels can hurt another.
- Validation gets harder: every retrain must be checked on three products' metrics at once.

Q Reframe it.

A Consolidation moves cost from many places to one, and moves risk from separation to interference. Measure the interference directly before accepting the savings.

Basis: The 5 billion images, 1,000 images a second and $2.50 an hour are illustrative; the pain statement is from the paper (source-confirmed); arithmetic recomputed.

02 Was it multi-task training, or just a better backbone?

EVIDENCE

SCENARIO

The paper's Table 4 shows the unified embedding beating each old specialized embedding: Shop-the-Look P@1 33.0 to 52.8, Flashlight avg P@20 53.4 to 60.2, Lens avg P@20 17.8 to 18.4. A colleague says: “That proves training one model on all three datasets is better than training three.”

HOW TO THINK ABOUT IT

Ask what else changed between the two things being compared. A fair test changes only the thing you want to credit.

Q What does Table 4 compare?

A The new unified model against the old production embeddings: gains of +19.8, +6.8 and +0.6 points (source-confirmed).

- The old embeddings came from older networks: VGG16 for Flashlight, ResNeXt50 for Lens, ResNet101 for Shop-the-Look. The paper notes that its architectures are fragmented.

Q Why can't multi-task training take the credit?

A Three things changed at once: the backbone (SE-ResNeXt101), the training data and the training method.

- Any of them could explain a gain. A colleague's claim needs a comparison where only the training method differs.

Q Which table isolates multi-task training?

A Table 3, which trains the same architecture on each dataset alone and on all three (source-confirmed).

- Shop-the-Look P@1: 49.2 alone, 52.8 with all, a gain of +3.6. Flashlight avg P@20: 53.4 to 60.2, +6.8. Lens avg P@20: 18.2 to 18.4, +0.2.
- So most of Shop-the-Look's 19.8-point jump over the old model came from the architecture and data (only 18% from multi-task). For Flashlight the single-dataset model matched the old 53.4, so on these numbers all of its gain came from the other datasets. For Lens, the gain is negligible.

Q What else would you want before trusting the +3.6?

A An idea of the noise. Shop-the-Look's evaluation set has 600 query objects, so a P@1 of 52.8 has a 95% margin of about ±4.0 points.

- Two single estimates 3.6 points apart on separate samples would not be distinguishable. Compare on the same queries (a paired comparison) and over several training runs.

Q Reframe it.

A When a headline comparison bundles several changes, find the experiment that changes only one. The result can differ a lot by product.

Basis: Table 3 and Table 4 values are from the paper (source-confirmed); the percentages and the margin are arithmetic (recomputed); the 600-query set size is from the paper.

03 Offline says +0.6, human raters say +110%

EVALUATION

SCENARIO

For Lens, the paper's offline evaluation moves from avg P@20 of 17.8 to 18.4. Human raters comparing the same two systems report a P@5 change of +110.1%, and the online test shows clickthroughs up 35%. A reviewer asks which number is right.

HOW TO THINK ABOUT IT

Different measurements answer different questions. Find what each one measures, on what data, and what it leaves out, before choosing which to believe.

Q How big is each change?

A Offline: +0.6 points on a base of 17.8, a 3.4% relative gain. Human: +110.1% in P@5. Online: close-ups +32.7%, repins +46.7% and clickthroughs +35.0% (source-confirmed).

- These cannot all be the same effect measured the same way. Relative changes on low bases also look large, so report absolute numbers alongside them.

Q Why might the offline number understate the gain?

A It measures the embedding alone, on a small corpus, with labels derived from user engagement, which is noisy.

- The Lens evaluation set uses about 1,000 camera queries against about 49,000 images, built from engagement and filtered with human judgment (source-confirmed).
- The paper says offline evaluations measure the embeddings alone, on a small corpus, while real retrieval systems are more complex.

Q What does the human judgment add?

A A different question: for a live query to the real product, are the 5 results shown relevant?

- It tests the whole retrieval path against a large corpus, with raters who know the criteria. It is closer to what the user sees, though raters are not users.

Q Which do you use for what?

A Offline to iterate quickly, human judgment to check relevance before exposure, online tests to confirm behavior when traffic and risk allow.

- Trust the later, more realistic signals for a launch decision, and use the earlier ones to find candidates.

Q Reframe it.

A When two measurements disagree, the disagreement is information about what each one covers. Do not pick the one you like, and do not average them.

Basis: Offline, human and online figures are from the paper's Table 4, Table 5 and Table 7 (source-confirmed); the evaluation set description is from the paper; the relative gain is arithmetic (recomputed).

04 Launch Shop-the-Look without an A/B test?

LAUNCH DECISION

SCENARIO

Shop-the-Look had not launched to any user when the unified embedding was ready, so no A/B test could be run. The team has human judgments. A director asks whether that is enough to ship.

HOW TO THINK ABOUT IT

A weaker signal is acceptable when a stronger one is not available, if you know exactly what it cannot tell you and you plan to get the stronger one later.

Q What evidence did the team have?

A Human raters judged P@5 over queries: 37.3% for the old specialized embedding and 64.2% for the unified one, a change of +72.1% (source-confirmed).

- The unified embedding was better in all categories but one. Candles fell from 60.0% to 40.0%.
- Offline P@1 on the curated set was also 52.8 against 33.0.

Q What can raters not tell you?

A What users will do. Raters judge relevance with a guide. Users click, save and buy for other reasons.

- The paper reports worker consistency of 98.0% for Flashlight and 92.9% for Lens, a test-retest check on the judging. It reports none for Shop-the-Look.
- Raters do not tell you about business metrics such as purchases, or about harm to specific segments.

Q Is it enough to ship?

A Yes, if the cost of being wrong is small and reversible, and you ship with controls.

- Staged rollout to a small share of users, the old path kept as a fallback, and the click and conversion metrics watched from the first hour.
- Look at the Candles category and others that lost or gained little.

Q What do you do afterward?

A Replace the missing A/B test as soon as there is traffic. Run it against the old path to get the causal read.

- Record the human-judgment result as the launch criterion and the A/B result as the confirmation.

Q Reframe it.

A Evidence quality is a spectrum. When you cannot get the best, name what the next-best misses and plan how to get the rest.

Basis: Human-judgment numbers and the missing A/B test are from the paper (source-confirmed); the rollout plan is reasoned.

PART 2

The model: shared backbone, classification heads and binary embeddings

How a classifier produces a similarity space, how the four tasks share one trunk, and what binarizing costs.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Shared backbone and head	A backbone is the network that turns an image into features. A head is a small task-specific layer on top. Here one backbone feeds four heads, so the expensive learning is shared.
Proxy-based metric learning	Training an embedding with a classifier. Each class has a proxy, a reference vector, and an image is pulled toward the proxy of its class and away from the others. The embedding before the head becomes a similarity space.
Softmax cross-entropy	The standard classification loss. It turns a score for each class into probabilities that add up to one, and penalizes a low probability on the correct class.
Proxy subsampling	With 15,000 or more classes, computing the loss over every proxy for every batch is slow. Each batch uses a sample of the proxies that always includes the classes present in the batch.
Binarization	Keeping one bit per dimension, by thresholding each number at zero: 1 if it is zero or more, 0 otherwise. It shrinks a 32-bit float embedding by 32 times.
GroupNorm	A normalization layer that rescales groups of channels within each example, independent of batch size. The paper chose it over LayerNorm as better suited to multi-task training.
GradNorm	A method that learns the weights of several task losses during training so no task dominates. The paper tried it and found equal weights gave the best balance.
Uniform dataset mixing	Each training batch takes an equal share from every dataset, regardless of dataset size, so small datasets are seen more often per epoch than large ones.
Where to read	Zhai et al.: Learning a Unified Embedding for Visual Search at Pinterest

05 Can a classifier train an embedding?

MECHANISM

SCENARIO

A teammate says: “Embeddings need a triplet or contrastive loss. A softmax classifier cannot give you a similarity space.” The paper trains its embedding with four classification heads.

HOW TO THINK ABOUT IT

Follow what the classifier forces the layer before it to do. Then compare the cost of the alternatives at the scale of the data.

Q What is a proxy, and what do the heads do?

A Each head is a fully connected layer whose weights are the proxies, one reference vector per class, followed by a softmax cross-entropy loss (source-confirmed).

- Four heads: Flashlight class, Shop-the-Look product class, Shop-the-Look instance class and Lens category class. The heads share the layers up to the embedding.

Q Why does that produce a useful similarity space?

A To classify well, images of the same class must land near the same proxy. The embedding layer feeding all four heads is therefore pushed to put visually similar images close together.

- Similarity is learned as a side effect of classification, and the shared layer has to be useful to four tasks at once.

Q Why not triplets?

A A triplet loss needs good triples of anchor, same-class and different-class images, and the number of possible triples grows far faster than the number of images. Finding informative ones is expensive.

- A classifier uses each image's label directly. The cost is that you need class labels and a reasonable set of classes, which the three datasets provide.

Q How does it cope with 15,000 or 50,000 classes?

A By subsampling proxies in each batch, so the softmax covers a sample and not every class.

- The sample always includes the classes present in the batch (source-confirmed). The paper's example: sampling 2,048 classes gives a worst-case loss from random guessing of ln(2048), about **7.62**.
- Sparse tensors avoid computing dense gradients for all the proxies.

Q Reframe it.

A A different objective can reach the same goal by a cheaper route. Ask what the layer in the middle must learn to make the final loss small.

Basis: The head design, subsampling and the 2,048-class example are from the paper (source-confirmed); the triplet-cost argument is reasoned; ln(2048) is arithmetic (recomputed).

06 A noisy head in a shared trunk

DEBUGGING

SCENARIO

A taxonomy change leaves 8% of Shop-the-Look product-class labels wrong (illustrative). In the next retrain, Flashlight's avg P@20 falls by 1.5 points, even though Flashlight's data, head and evaluation set did not change.

HOW TO THINK ABOUT IT

A shared component is a shared risk. Work out how a change in one place reaches another, then design a test that separates the cause from the coincidences.

Q Why would Flashlight move?

A The trunk is shared. Gradients from the noisy Shop-the-Look head update the same layers that produce Flashlight's embeddings.

- Mislabeled images pull the shared embedding toward the wrong categories, which is negative transfer in action.

Q How do you confirm it is the cause?

A An ablation: retrain with the Shop-the-Look product head removed, or with the labels cleaned, and compare Flashlight's metric.

- Evaluate each head's metric before and after, since an average would hide the move. Control for training noise with a repeated run, because a 1.5-point change may partly be randomness.

Q What do you do about it?

A Fix the labels first. While that happens, down-weight or pause the affected head instead of shipping the regression.

- The paper ignores gradients from tasks that an image has no label for, so each dataset only affects heads it has labels for. That limits cross-contamination without removing it.

Q How would you have caught it earlier?

A A per-product regression check in every retrain. The release gate is each product's metric, not an overall score.

- A label-quality audit on a sample of each dataset before it enters training.

Q Reframe it.

A In a shared design, a problem in one part shows up as a symptom in another. Monitor each part separately and keep a way to isolate any one of them.

Basis: The mislabeled share and the 1.5-point drop are illustrative; the shared-trunk mechanism and gradient masking are from the paper (source-confirmed); the diagnosis is reasoned.

07 Float, low-dimensional float, or binary?

DESIGN CHOICE

SCENARIO

To cut storage, someone proposes keeping float32 embeddings but shrinking them to 64 dimensions. Another proposes thresholding the 2,048-dimensional embedding to bits. A third says binarizing always costs accuracy, so keep floats.

HOW TO THINK ABOUT IT

Compare the options at the same storage cost, then look at what the paper measured, not at what it is plausible to believe.

Q How do the options compare in storage?

A A 2,048-dimension binary embedding is 256 bytes. That is the same storage as 256 ÷ 4 = **64 float32 values**.

- A full 2,048-dimension float embedding is 8,192 bytes, 32 times larger.
- So the real comparison at equal storage is 2,048 bits against 64 floats.

Q What did the paper's earlier work find?

A For the same storage cost, a learned binary embedding performed better than a learned low-dimensional float one (source-confirmed).

- That is the case for binary over “shrink the floats.”

Q Does binarizing cost accuracy?

A It did here, at first. The paper's Table 1: float baseline 47.5, 60.1 and 18.6; the same model binarized 41.9, 55.6 and 17.7 (Shop-the-Look P@1, Flashlight and Lens avg P@20).

- That is a loss of 5.6, 4.5 and 0.9 points, which the paper calls “a significant drop in performance between raw float and binary features.”

Q What fixed it?

A Changing the architecture around the binary layer: softmax without the L2-normalization constraint, GroupNorm, ReLU and dropout.

- The final binary model scored 52.8, 60.2 and 18.4: 5.3 above the float baseline on Shop-the-Look, 0.1 above on Flashlight and 0.2 below on Lens.
- So the third person is half right. Binarizing costs accuracy unless the model is adapted to it.

Q How do you decide on your data?

A Compare at equal storage on your own evaluation set, with the binary version trained as such, and repeat the check after every retrain.

Q Reframe it.

A A compression choice is a claim about accuracy, so measure it. “Always” and “never” are rarely right when the model can adapt.

Basis: Table 1 values and the earlier-work claim are from the paper (source-confirmed); the storage comparison is arithmetic (recomputed).

08 Equal loss weights and uniform dataset mixing

TRAINING DESIGN

SCENARIO

The three datasets have very different sizes: about 800K, 540K and 340K images. The paper mixes them equally in every batch and gives the four losses equal weights. A reviewer says: “Weighting by dataset size or learning the weights would obviously be better.”

HOW TO THINK ABOUT IT

Test the intuition on the paper's own setup. Work out what uniform mixing does to the small datasets, and ask what evidence would show the intuition is right.

Q What did the paper do, and try?

A Every batch takes a uniform mix of the datasets, an epoch is defined by one pass through the largest, and the four task losses have equal weights (source-confirmed).

- It also tried sampling by dataset size and learning the loss weights with GradNorm, and reports that the simple approach gave the best balance of performance.

Q What does uniform mixing mean for the small datasets?

A They are repeated. In one epoch of 800K Flashlight images, Lens sees 800 ÷ 540 = 1.5 passes and Shop-the-Look sees 800 ÷ 340 = 2.35.

- Over the paper's 9 epochs, Shop-the-Look is seen about 21 times and Lens about 13. Repetition raises the risk of overfitting the small sets.

Q Why might equal loss weights be wrong in general?

A The heads differ: a softmax over 15K classes and one over 189 produce losses of different size, and tasks learn at different speeds.

- GradNorm exists to balance this. That equal weights worked here is an empirical result for these tasks, not a rule.

Q How would you test the reviewer's claim on your data?

A Train under each scheme and compare each product's metric, watching the weakest product and not the average.

- Repeat runs, since differences between schemes may be within noise.

Q Reframe it.

A A simple baseline that wins is a result worth keeping. Before adding machinery, check it against the baseline on the metric that matters per task.

Basis: Mixing, loss weighting and the alternatives tried are from the paper (source-confirmed); the dataset sizes are from the paper; the pass counts are arithmetic (recomputed).

PART 3

The index and the query path: storage, shards and search

Sizing a binary index, checking a scan against the hardware, and what scatter-gather does to latency and to resharding.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Hamming distance and popcount	For two bit strings of equal length, the number of positions where they differ. Compute it by XOR-ing the strings and counting the 1 bits, which a processor can do in one instruction (popcount).
Shard and replica	A shard is one slice of the index, holding a share of the rows. A replica is a full copy of a shard that serves queries. Shards add capacity, and replicas add throughput and availability.
Scatter-gather	Sending one query to every shard in parallel, then merging each shard's best results into one final list. It is exact when each shard returns its own top k.
Approximate nearest-neighbor (ANN) index	A structure that finds close vectors without comparing against every stored one, trading a little recall for a large speed-up.
Consistent hashing	A way to place rows on shards so that adding or removing one shard moves only about 1 divided by the new shard count of the rows, instead of nearly all of them.
Tail latency and fan-out	A query that waits for N parallel calls is as slow as the slowest call. The more calls, the more likely at least one is slow, so the query's tail is worse than any shard's.
Memory bandwidth	How many bytes per second a machine can read from memory. A scan that must read the whole shard cannot run faster than shard size divided by this bandwidth.
Embedding version	A column recording which model produced a row's embedding, so old and new vectors can coexist during a backfill and a rollout can be staged or undone.
Where to read	Zhai et al.: Learning a Unified Embedding for Visual Search at Pinterest

09 Size the index

SIZING

SCENARIO

A storage engineer asks how much space the visual search index needs, and whether the shift from the old embeddings to the unified binary one changes the plan. Assume 5 billion images and a 2,048-dimension unified embedding (illustrative).

HOW TO THINK ABOUT IT

Compute the size in each format, name which assumption each number depends on, and then look for what the plan leaves out.

Q How big is the index in each format?

A Per image: 2,048 dimensions × 4 bytes = 8,192 bytes as float32, or 2,048 ÷ 8 = 256 bytes as bits.

- Across 5 billion images: **40.96 TB** as float and **1.28 TB** binarized, 32 times smaller.
- For comparison, a 4,096-dimension float embedding like Flashlight's raw one would be 16,384 bytes an image, 81.92 TB, 64 times the binary size.

Q How many shards?

A With a target of at most 100 GB a shard: 1.28 TB ÷ 100 GB = 12.8, so 13 at the bare minimum. With 20% headroom, 1.536 TB ÷ 100 GB = 15.36, so **16 shards**.

- The headroom covers growth and the replication overhead of each shard's node.

Q What does the 32 times depend on?

A Only the bit width: 32 bits down to 1. It does not depend on the corpus size or the dimension.

- A 10-billion-image corpus doubles both totals and leaves the ratio alone. The 5 billion and the 2,048 are assumptions. The paper gives neither.

Q What does the plan leave out?

A Several things.

- **Two versions at once.** During a backfill both the old and the new embeddings exist, so the binary index needs about 2.56 TB while the rollout is staged.
- The search structure itself (Question 13), replicas for availability and metadata.

Q Reframe it.

A A size estimate is a product of assumptions. State them, show which ones the answer is sensitive to, and add the transient states, such as a migration, that peak above the steady state.

Basis: The 5 billion images, the 2,048 dimensions and the 100 GB shard target are illustrative; the 4,096-dimension old embedding is from the paper (source-confirmed); the sizes are arithmetic (recomputed).

10 Review the per-shard scan

CODE REVIEW

SCENARIO

An engineer ships the function below to search one shard of 312.5 million 256-byte codes. In staging, the process runs out of memory, and when it does not, a query takes tens of seconds. The code gives correct results on a test of 1,000 rows.

```
import numpy as np

def shard_top_k(query_bits, corpus_bits, k=20):
    # query_bits: (256,) uint8; corpus_bits: (N, 256) uint8
    xor = np.bitwise_xor(corpus_bits, query_bits)
    distances = np.unpackbits(xor, axis=1).sum(axis=1)
    return np.argsort(distances)[:k]
```

HOW TO THINK ABOUT IT

Compute the size of every intermediate array at the real data size, then think about what the last line does to a list that long.

Q What is wrong with the memory use?

A np.unpackbits turns every byte into 8 bytes, so the temporary array is N × 2,048 bytes.

- For one million rows that is about 2 GB. For the whole shard, 312.5 million rows, it is about **640 GB**, eight times the shard itself. The 1,000-row test hid this.

Q What is wrong with the time?

A Two things. A full scan must read the shard: 80 GB at an assumed 100 GB a second is at least 0.8 s.

- np.argsort sorts all 312.5 million distances to keep 20 of them, a job that needs a selection, not a sort. argpartition finds the k smallest in linear time.

Q State the fix.

A Work in chunks, count bits with a 256-entry lookup table, select the top k of each chunk with a partition and merge with a running top k.

- Memory is bounded by the chunk size, not the shard size. The lookup replaces the 8-times expansion with an array the size of the chunk.

Q Does the fix make exact scanning viable at 926 queries a second?

A No. Each query still reads 80 GB per shard, so one replica serves about 1.25 queries a second, and the peak needs about 740 replicas per shard (Question 13).

- Keep the scan as a correctness reference to measure an approximate index against, and for re-ranking a short candidate list.

Q What would you test?

A Identical codes give distance 0 and bit-complements give 2,048. Run chunk boundaries at sizes that do not divide N. Compare against the slow version on a sample, and test a shard with fewer than k rows.

Q Reframe it.

A Run the numbers at production scale before shipping. A function that is right on a thousand rows can be unusable on three hundred million.

REFERENCE CODE

```
POP = np.array([bin(i).count("1") for i in range(256)], dtype=np.uint8)

def shard_top_k(query, corpus, k=20, chunk=1_000_000):
    best_d = np.empty(0, dtype=np.int32)
    best_i = np.empty(0, dtype=np.int64)
    for start in range(0, len(corpus), chunk):
        block = corpus[start:start + chunk]
        d = POP[np.bitwise_xor(block, query)].sum(axis=1, dtype=np.int32)
        idx = np.argpartition(d, min(k, len(d) - 1))[:k]
        best_d = np.concatenate([best_d, d[idx]])
        best_i = np.concatenate([best_i, idx + start])
        keep = np.argsort(best_d)[:k]
        best_d, best_i = best_d[keep], best_i[keep]
    return best_i, best_d
```

Basis: The shard size is from the assumptions of Question 9 (illustrative); the 100 GB a second is illustrative; the memory and time arithmetic is recomputed; the fix was checked by hand and not run on a shard of that size.

11 Add a 17th shard

RESHARDING

SCENARIO

The index has outgrown 16 shards. The plan is to add a 17th. Rows are placed by image_id modulo the number of shards. An engineer says the move is cheap because only one shard is new.

HOW TO THINK ABOUT IT

Work out how many rows change shard under the placement rule, price the move in bytes, and think about what a query sees while rows are in transit.

Q What moves under modulo placement?

A A row stays only if image_id gives the same remainder modulo 16 and modulo 17. Only about 1 row in 17 does, so about 16 ÷ 17 = **94%** of rows move.

- Almost every row's home changes, not just the new shard's share.

Q What would consistent hashing move?

A Only the rows the new shard takes over: about 1 ÷ 17 = **6%**.

- The cost in bytes: 1.28 TB × 16 ÷ 17 = about 1.2 TB for modulo, against 1.28 TB ÷ 17 = about 75 GB for consistent hashing.

Q What does a query see during the move?

A Queries go to every shard, so a row in transit may be on two shards, or on neither for a moment.

- Copy before deleting, so a row is always somewhere. Deduplicate by image_id when merging the shards' results.
- Stage the move shard by shard, using the embedding version and a routing table, so it can be paused or reversed.

Q Does placement matter for query correctness?

A Not for scatter-gather: every shard is searched, so any placement gives the same answer. It matters for the cost of changing the shard count.

- That is why a system with recurring growth and backfills should pay for consistent hashing, while one with a fixed shard count can use modulo.

Q Reframe it.

A A placement rule is a decision about the cost of change. Ask what happens to the data when the number of places changes, not just where it sits today.

Basis: The 16 and 17 shards and the 1.28 TB index are illustrative (Question 9); the remap fractions and byte counts are arithmetic (recomputed); consistent hashing is from Lesson 5, Concept 1.

12 Fan-out makes the tail worse

SCATTER-GATHER

SCENARIO

Each shard answers within 100 ms 99% of the time. A query fans out to all 16 shards and waits for every one. Users complain that search is often slower than 100 ms, though no shard's dashboard shows a problem.

HOW TO THINK ABOUT IT

A query is as slow as its slowest part. Compute the chance that at least one of the parts is slow, and see how it changes with the number of parts.

Q What fraction of queries are slow?

A A query is under 100 ms only if all 16 shards are. If shards are independent, that is 0.99^16 = 0.85, so **about 15%** of queries take longer than 100 ms.

- The shards' own p99 becomes roughly the query's p85.

Q How does it grow?

A With 154 shards, the 10 times larger corpus of Question 17, the chance is 1 − 0.99^154 = **79%**.

- More shards make the tail worse unless each shard gets faster. Real shards are not independent, so the true numbers differ, but the direction holds.

Q What can you do about it?

A Hedge, bound, or shrink the problem.

- **Hedged requests:** if a shard has not answered by its own p95, send the same request to another replica and take the first answer. It adds about 5% more shard requests.
- **Timeouts with partial results:** return the best results from the shards that answered, accepting a small recall loss, and monitor how often it happens.
- **Fewer, bigger shards or faster shards:** reduce the fan-out or the shard tail.

Q What would you monitor?

A Per-shard p99, the maximum latency across shards for each query, the share of queries that used a hedge or a timeout, and the fan-out.

Q Reframe it.

A Waiting for several things multiplies their chances of being slow. Compute the tail of the whole, not the tail of one part.

Basis: The 100 ms, the 99% and the independence are illustrative; the tail probabilities are arithmetic (recomputed); hedging is a standard technique (reasoned).

13 How many replicas at peak?

CAPACITY

SCENARIO

The index is 16 shards chosen by capacity. Peak traffic is 926 searches a second. A colleague says 16 machines are enough because each holds 80 GB of codes. Another proposes brute-force scanning, because binary codes are cheap to compare.

HOW TO THINK ABOUT IT

Every query touches every shard. Work out what one replica can do, then what that means for the number of machines, with and without a search index.

Q What load does each shard see?

A All of it. Scatter-gather sends every query to every shard, so each shard serves 926 queries a second.

- Capacity decides the shard count. Throughput decides the replicas per shard.

Q What does the brute-force scan need?

A Each query reads the shard's 80 GB. At an assumed 100 GB a second that is 0.8 s, so one replica serves 1.25 queries a second.

- 926 ÷ 1.25 = about **740 replicas per shard**, and 740 × 16 = about **11,850** machines.

Q What does an index change?

A Suppose an approximate index answers a shard query in 5 ms (illustrative), so one replica serves 200 queries a second.

- 926 ÷ 200 = 4.6, so 5 replicas per shard and **80** machines in all. That is roughly a 150-times difference, and it is the reason an index is mandatory at this scale.

Q What must you measure for the index?

A The cost of approximation. Recall against the exact scan on a sample: of the true top 20, how many does the index return? Also latency at p99, memory use and build time.

- The exact scan is the reference for measuring recall, which is its best use.

Q Reframe it.

A Sizing by storage and sizing by traffic give different answers. Compute both, and let the larger one decide.

Basis: The 100 GB a second and the 5 ms per shard query are illustrative; the replica counts are arithmetic (recomputed); the paper does not describe Pinterest's index.

PART 4

Evaluation, operations, risk and scale

Shipping a new embedding safely, building evaluation sets you can trust, what to protect, and what ten times the size breaks.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Staged rollout	Moving traffic to a new version in steps, such as one shard or a few percent of users at a time, with the old version kept ready, so a problem is limited and reversible.
Evaluation (golden) set	A fixed set of queries with known correct results, used to score every model change on the same footing.
Confidence interval	A range that the true value probably lies in, given a sample. For a share p measured on n queries, the 95% margin is about 1.96 times the square root of p(1−p)/n, so it shrinks only with the square root of n.
Paired comparison	Scoring two systems on the same queries and looking at the differences, which removes most of the query-to-query variation and needs fewer queries.
Distractors	The other items in the corpus that could be returned instead of the right one. A larger corpus has more of them, so precision measured on a small corpus overstates precision at full scale.
Cold start	The gap between a new item arriving and being searchable, caused by the time to embed it and write it to the index.
Retention	How long data such as query images and their embeddings is kept, and who can access it.
Near-duplicate and adversarial images	Images that are almost copies of each other, or crafted so that their embedding lands near a chosen target, which can crowd or hijack search results.
Where to read	Zhai et al.: Learning a Unified Embedding for Visual Search at Pinterest

14 A retrain ships and binary quality drops

OPERATIONS

SCENARIO

A new training run improves the float embedding on every product's evaluation set. The team is ready to start a full backfill of the binary index. A reviewer asks what has been checked on the binary version.

HOW TO THINK ABOUT IT

A change measured in one format does not carry over automatically to another. Decide what must be re-measured, how to ship in steps, and what undoing costs.

Q What should be checked before the backfill?

A The binary embedding on each product's own evaluation set, compared with the one in production.

- The paper's Table 1 shows how much can be lost: a binarized baseline scored 5.6, 4.5 and 0.9 points below the same model in float (source-confirmed). A float improvement may not survive thresholding.

Q How do you ship it in steps?

A Write the new vectors with a new embedding version next to the old ones, then shift traffic gradually, shard by shard, keeping the old index live.

- Rollback is then a pointer change, because the old index was never deleted. The cost is storage: about 1.28 TB extra while both versions exist.

Q What does redoing a backfill cost?

A About 1,389 GPU-hours at the assumed speed (Question 1). With 100 GPUs that is about 14 hours, and about $3,472.

- A bad launch costs more, so a staged rollout with a real gate is cheap insurance.

Q What gates the rollout?

A A rule written in advance: no product's metric may fall below its current value by more than a stated margin, on the binary embedding, in the offline set and in human judgment on live queries.

- Use the weakest product, not the average (Question 6).

Q Reframe it.

A Re-measure in the form that ships. Keep the old version running until the new one has proved itself in that form.

Basis: Table 1 values are from the paper (source-confirmed); the backfill speed and price are illustrative; the rollout procedure is reasoned; arithmetic recomputed.

15 Can you trust the evaluation set?

EVALUATION

SCENARIO

A new embedding raises Shop-the-Look P@1 from 49.2 to 52.8 on the paper's evaluation set of 600 query objects searched in a corpus of 51,421 images. The team wants to ship on that result.

HOW TO THINK ABOUT IT

Ask how much noise a number of that size carries, and whether a small corpus can say anything about a corpus of billions.

Q What are the sets?

A Shop-the-Look: 600 curated objects with 1,421 ground-truth matches, searched in 51,421 images (50K sampled plus the matches). Flashlight: 807 queries against 42,881 images. Lens: about 1,000 queries against about 49,000 images (source-confirmed).

Q How noisy is a P@1 on 600 queries?

A The 95% margin for 0.528 on 600 queries is $1.96 \times \sqrt{(0.528 \times 0.472 \div 600)}$ = about **±4.0 points**.

- For two systems scored on separate samples, the margin on the difference is about ±5.7 points, larger than the 3.6-point gain.
- Scoring both on the same 600 queries (paired) removes much of that, but the paper does not show a significance test for this comparison.

Q What does a small corpus hide?

A Distractors. With 51,421 images the right answer competes with few lookalikes. With billions, many more images look similar, and precision falls.

- Offline numbers rank models. They do not predict production precision.

Q How do you keep the sets useful?

A Keep one frozen set for trend lines and refresh a second from real traffic, labeled with a written guide.

- Add the categories and image types users search for now, and check label agreement.

Q Reframe it.

A A metric is a sample. Know its margin and what population it stands for before deciding on a difference that is smaller than the margin.

Basis: The set sizes and both P@1 values are from the paper (source-confirmed); the margins are arithmetic (recomputed); the observation about distractors is reasoned.

16 Camera photos as queries: privacy and abuse

RISK

SCENARIO

Lens lets users point a phone at anything and search with the photo. A security reviewer asks what the system stores, who can see it, and how it could be abused. The paper does not discuss these.

HOW TO THINK ABOUT IT

List what is collected and kept, then ask for each how long it lives and who can read it. For abuse, ask how an uploader could push a result up or crowd others out.

Q What is sensitive here?

A The queries. Camera photos can show people, homes, documents and screens, and the embedding of a photo may still encode content.

- The index images are public Pins. The queries are private moments.

Q What should the policy be?

A Keep query images only as long as needed to serve and debug, delete them on a schedule, and restrict access to logs and stored embeddings.

- State what is stored in the privacy notice, and exclude query images from training unless users agree.

Q How could the index be abused?

A Spam and near-duplicate uploads can crowd results with copies of one image. A crafted image can be made to embed near a popular query so it appears in many searches.

- The second is a risk of the same kind as poisoning attacks on retrieval systems, applied to images (reasoned).

Q What defenses help?

A Deduplicate near-copies at ingest. Watch for dense clusters of new embeddings around popular queries. Limit uploads per account and review reports.

- Track result diversity as a quality metric, so a crowd of copies shows up as a drop.

Q Reframe it.

A A system that accepts content from the public takes on the risks of that content, and the people searching bring their own private data. Decide what you keep and who can write.

Basis: The paper does not cover privacy or abuse; this question is reasoned, and nothing in it was tested.

17 Ten times the images and the traffic

SCALE

SCENARIO

Imagine the corpus grows to 50 billion images and peak traffic to 9,259 searches a second. A manager asks what breaks first, and whether to add machines or change the design.

HOW TO THINK ABOUT IT

Scale each quantity by ten and see which one stops being a matter of adding machines. Then pick the lever for each limit.

Q What does ten times do to storage and shards?

A The binary index is 12.8 TB. At 100 GB a shard with 20% headroom that is 12.8 × 1.2 ÷ 0.1 = 153.6, so **154 shards**.

- Storage is easy: more shards. The float index would be 410 TB.

Q What does it do to the query path?

A Exact scanning is out: each shard is as big as before but there are ten times as many queries per second and ten times as many shards to ask.

- An approximate index is mandatory (Question 13). Fan-out is the next limit: with 154 shards and a 99% shard p99, the chance that at least one is slow is 1 − 0.99^154 = 79% (Question 12).

Q What does it do to the backfill?

A 50 billion ÷ 1,000 per GPU-second = 13,889 GPU-hours, about **$34,722** a pass, ten times as much.

- That makes the single shared model more valuable, and the staged rollout and gates (Question 14) more important.

Q Which lever for the fan-out?

A Route each query to a subset of shards with a coarse first step, such as a clustering of the codes, instead of asking all 154.

- It cuts cost and tail, and can miss a result on a shard that was skipped. Measure that recall loss against the full fan-out before adopting it.

Q Reframe it.

A Some limits yield to more machines and some do not. Find the ones that do not, here query compute and fan-out, and change the design for those.

Basis: The 50 billion images and 9,259 searches a second are ten times the illustrative baseline; every figure is arithmetic (recomputed) from earlier questions' assumptions.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #1. The one source is the Pinterest paper, whose full text was read on 2026-10-17. The paper describes the model, the data and the evaluation. It does not describe Pinterest's index, sharding or serving system, so everything about those, and every figure marked illustrative, is a design argument made for this book and was not run on real systems.

- Zhai, Wu, Bai, Lan, Feiszli et al.: Learning a Unified Embedding for Visual Search at Pinterest, KDD 2019 (arXiv 1908.01707), full text read 2026-10-17
https://arxiv.org/abs/1908.01707